

Отчёт по лабораторной работе №1

Computer Skills for Scientific Writing

Филибер Никола

В лабораторной работе выполнена подготовка рабочего окружения для курса. Настроена система контроля версий Git, созданы SSH- и PGP-ключи, настроено автоматическое подписание коммитов, подготовлены учётные записи GitVerse и SourceCraft. Установлены и настроены инструменты, необходимые для работы с репозиторием и семантическим версионированием. На основе предоставленного шаблона создано рабочее пространство курса, настроен Git Flow и сформирован первый релиз проекта версии 1.0.0.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
2	Теоретическая часть	8
2.1	Система контроля версий Git	8
2.2	SSH	8
2.3	PGP и подписание коммитов	8
2.4	Git Flow	9
2.5	Семантическое версионирование	9
2.6	Общепринятые коммиты	9
3	Выполнение лабораторной работы	11
3.1	Настройка Git	11
3.2	Настройка GitVerse и SourceCraft	11
3.3	Создание SSH-ключа	12
3.4	Создание PGP-ключа	12
3.5	Настройка автоматического подписания коммитов	13
3.6	Установка дополнительных инструментов	13
3.7	Создание репозитория курса	14
3.8	Инициализация структуры курса	14
3.9	Добавление SourceCraft	15
3.10	Настройка Git Flow	16
3.11	Создание релиза 1.0.0	16
3.12	Завершение релиза	17
3.13	Подготовка данных релиза	17
4	Результаты	19
5	Заключение	20
	Список литературы	21

Список иллюстраций

Список таблиц

1 Введение

В данной лабораторной работе выполняется подготовка рабочего пространства. Основное внимание уделяется настройке Git, SSH и PGP, работе с платформами GitVerse и SourceCraft, организации структуры каталогов курса и использованию Git Flow.

1.1 Актуальность

Подготовка такого окружения необходима для дальнейшего выполнения лабораторных и научно-исследовательских работ в рамках курса.

1.2 Цель работы

Цель лабораторной работы — подготовить и настроить рабочее окружение для выполнения лабораторных работ по курсу

1.3 Задачи

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Настроить систему контроля версий Git.
2. Создать SSH-ключ для доступа к удалённым Git-репозиториям.
3. Добавить SSH-ключ в GitVerse и SourceCraft.

4. Создать PGP-ключ для подписания коммитов.
5. Добавить публичный PGP-ключ в GitVerse и SourceCraft.
6. Настроить автоматическое подписание Git-коммитов.
7. Установить необходимые средства разработки.
8. Установить Node.js и средства работы с общепринятыми коммитами.
9. Установить и настроить Git Flow.
10. Создать репозиторий курса на основе предоставленного шаблона.
11. Инициализировать структуру рабочего пространства курса.
12. Создать удалённую копию репозитория в SourceCraft.
13. Настроить Git Flow для созданного репозитория.
14. Создать первый релиз проекта версии 1 . 0 . 0.
15. Сформировать журнал изменений CHANGELOG . md.
16. Установить Quarto

...

2 Теоретическая часть

2.1 Система контроля версий Git

Git — распределённая система контроля версий, используемая для хранения истории изменений проекта.

В данной работе Git используется для хранения файлов курса, создания коммитов, организации веток и тегов, а также синхронизации проекта с GitVerse и SourceCraft.

2.2 SSH

SSH используется для защищённого взаимодействия с удалёнными Git-репозиториями без необходимости вводить пароль при каждой операции.

Для работы создаётся отдельная пара ключей Ed25519. Закрытый ключ хранится на локальном компьютере, а открытый ключ добавляется в учётные записи GitVerse и SourceCraft.

2.3 PGP и подписание коммитов

PGP используется для криптографического подтверждения авторства коммитов. В рамках лабораторной работы создаётся RSA-ключ размером 4096 бит.

Подписанный коммит позволяет платформе хостинга репозитория проверять, что подпись была создана ключом, связанным с пользователем.

2.4 Git Flow

Git Flow представляет собой модель организации веток Git. В рамках данной лабораторной работы используются основные ветки `master` и `develop`, а для подготовки версии создаётся временная ветка `release`.

Работа над первым релизом выполняется в ветке:

```
release/1.0.0
```

После завершения релиза Git Flow объединяет изменения с основными ветками и создаёт соответствующий тег версии.

Для тегов используется префикс `v`, поэтому версия `1.0.0` представляется тегом:

```
v1.0.0
```

2.5 Семантическое версионирование

При формировании релизов используется семантическое версионирование. Номер версии состоит из трёх основных компонентов:

```
MAJOR.MINOR.PATCH
```

Первый созданный в работе релиз имеет номер:

```
1.0.0
```

Для автоматизации работы с версиями и журналом изменений используются Node.js-инструменты.

2.6 Общепринятые коммиты

Общепринятые коммиты (Conventional Commits) задают единый формат сообщений Git-коммитов. Такой подход делает историю проекта более

структурированной и позволяет автоматизировать формирование журнала изменений.

В работе, например, используются сообщения:

```
feat(main): make course structure
```

и

```
chore(site): add changelog
```

Для поддержки такого рабочего процесса используются `commitizen`, `cz-customizable` и средства формирования журнала изменений.

3 Выполнение лабораторной работы

3.1 Настройка Git

Сначала была выполнена базовая настройка Git. Были указаны имя и адрес электронной почты пользователя:

```
git config --global user.name "Иванов Иван Иванович"  
git config --global user.email "<email>"
```

Была настроена корректная работа с UTF-8:

```
git config --global core.quotepath false
```

В качестве начальной ветки была установлена ветка master:

```
git config --global init.defaultBranch master
```

Также были настроены параметры обработки окончаний строк:

```
git config --global core.autocrlf input  
git config --global core.safecrlf warn
```

3.2 Настройка GitVerse и SourceCraft

Для хранения репозитория были подготовлены учётные записи на платформах GitVerse и SourceCraft.

GitVerse используется как один из удалённых серверов проекта, а SourceCraft был добавлен как дополнительный удалённый репозиторий.

3.3 Создание SSH-ключа

Для доступа к удалённым Git-репозиториям был создан SSH-ключ типа Ed25519:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519-git -C "<email>"
```

После создания ключ был добавлен в SSH Agent:

```
ssh-add ~/.ssh/id_ed25519-git
```

Публичная часть SSH-ключа была добавлена через веб-интерфейсы GitVerse и SourceCraft.

В результате появилась возможность выполнять операции `clone`, `fetch`, `pull` и `push` через SSH.

3.4 Создание PGP-ключа

Для подписания Git-коммитов был создан PGP-ключ:

```
gpg --full-generate-key
```

При генерации был выбран тип ключа RSA, размер ключа 4096 бит и указаны данные владельца ключа.

Для получения информации о созданных секретных ключах была использована команда:

```
gpg --list-secret-keys --keyid-format LONG
```

Из результата был получен отпечаток PGP-ключа.

Публичная часть ключа была экспортирована:

```
gpg --armor --export <PGP_FINGERPRINT>
```

Полученный публичный ключ был добавлен в настройки GitVerse и SourceCraft.

3.5 Настройка автоматического подписания

КОММИТОВ

Git был настроен на использование созданного PGP-ключа:

```
git config --global user.signingkey <PGP_FINGERPRINT>  
git config --global commit.gpgsign true  
git config --global gpg.program $(which gpg)
```

В процессе настройки была отдельно проверена работа GPG и механизм pinentry. Для корректной передачи терминала GPG была настроена переменная GPG_TTY.

После настройки Git получил возможность автоматически подписывать создаваемые коммиты.

3.6 Установка дополнительных инструментов

Для работы с Conventional Commits и семантическим версионированием была подготовлена среда Node.js.

Для macOS Node.js устанавливается с помощью Homebrew:

```
brew install node
```

Для управления ветками по модели Git Flow был установлен Git Flow:

```
brew install git-flow
```

Также были подготовлены инструменты для работы с общепринятыми коммитами и версиями проекта.

3.7 Создание репозитория курса

На GitVerse был открыт репозиторий-шаблон:

```
dharm/course-directory-student-template
```

На его основе с помощью функции «Использовать как шаблон» был создан новый публичный репозиторий:

```
2026-2-study-practical-scientific-writing
```

После создания репозиторий был клонирован в рабочее пространство курса.

Структура каталога была подготовлена в соответствии с методическими указаниями:

```
~/work/study/2026-2/2026-2==study-practical-scientific-writing/
```

3.8 Инициализация структуры курса

После клонирования был выполнен переход в каталог репозитория.

Название курса было записано в файл COURSE:

```
echo practical-scientific-writing > COURSE
```

Затем была выполнена подготовка структуры:

```
make prepare
```

Созданные изменения были добавлены в Git:

```
git add .  
git commit -am 'feat(main): make course structure'  
git push
```

3.9 Добавление SourceCraft

На платформе SourceCraft был создан репозиторий с тем же названием:

```
2026-2-study-practical-scientific-writing
```

Он был добавлен к локальному Git-репозиторию как дополнительный remote:

```
git remote add src ssh://ssh.sourcecraft.dev/<owner>/2026-2-study-p
```

После этого ветка `master` была отправлена в SourceCraft:

```
git push src master
```

Таким образом, один локальный репозиторий был связан с двумя удалёнными хранилищами.

3.10 Настройка Git Flow

В созданном репозитории был инициализирован Git Flow:

```
git flow init
```

Для тегов релизов был установлен префикс:

```
v
```

После инициализации была проверена текущая ветка:

```
git branch
```

Работа продолжилась в ветке `develop`.

Ветки репозитория были отправлены на удалённый сервер:

```
git push -u --all
```

3.11 Создание релиза 1.0.0

Для подготовки первого релиза была создана `release`-ветка:

```
git flow release start 1.0.0
```

В результате Git Flow создал ветку:

```
release/1.0.0
```

После этого был сформирован журнал изменений:

```
standard-changelog --first-release
```

Файл CHANGELOG.md был добавлен в индекс и зафиксирован:

```
git add CHANGELOG.md  
git commit -am 'chore(site): add changelog'
```

3.12 Завершение релиза

После подготовки журнала изменений релиз был завершён:

```
git flow release finish 1.0.0
```

При завершении Git Flow изменения из release-ветки были интегрированы в основные ветки проекта, а для версии был создан соответствующий Git-тег.

После этого изменения были отправлены на удалённый сервер:

```
git push --all  
git push --tags
```

Отдельно была выполнена отправка веток и тегов в настроенный репозиторий SourceCraft.

Наличие тегов на удалённых серверах проверялось через веб-интерфейсы GitVerse и SourceCraft.

3.13 Подготовка данных релиза

Для материалов релиза был создан отдельный каталог:

```
mkdir -p ../release
```

Журнал изменений был скопирован в него:

```
cp CHANGELOG.md ../release
```

На GitVerse релиз версии v1 . 0 . 0 был создан вручную через веб-интерфейс на основе опубликованного Git-тега.

4 Результаты

В результате выполнения лабораторной работы было подготовлено рабочее окружение курса «Computer Skills for Scientific Writing».

5 Заключение

В ходе выполнения лабораторной работы была достигнута поставленная цель — подготовлено рабочее окружение для выполнения лабораторных работ по курсу

Список литературы